

TENTAMEN

Kurs, kursnummer	HI1027, Objektorienterad programmering
Moment:	TEN1, 3,5 hp
Program: Åk:	TIDAA, TIELA, TIMEL -
Examinator:	Anders Lindström
Rättande lärare:	Anders Lindström
Datum: Tid:	2025-03-12 14:00-19:00
Hjälpmedel:	IntelliJ och Netbeans finns på tentamenskontot API-dokumentation och en kort syntaxlista finns i mappen P/Java/api/index.html
Omfattning och betygsgränser:	För betyget E krävs minst hälften av poängen på A-delen samt och hälften av poängen på B-delen samt att minst en av de två första programmeringsuppgifterna (9A och 10A) är helt korrekt lösta. B-delen rättas endast om minst hälften av poängen på A-delen uppnåtts. För betyg C krävs totalt minst 70 % av totala poängen; för betyg A krävs minst 90 % av totala poängen.
Övrig information:	Del A, Teoriuppgifter: Svara på separat papper. Del B, Programmeringsuppgifter: Lösningar lämnas på tentamenskontot. <i>När du skapar projektet i IntelliJ måste du kontrollera att projektet hamnar under enheten H, på ditt tentamenskonto. Alla lösningar skrivs i ett och samma projekt, men varje uppgift ska ligga i ett separat paket, med namnen uppg 9, uppg10, ...</i>

Mac-användare: För att komma åt måsvingar, hakparenteser och " | ", använd "AltGr" + lämplig tangent, t.ex. "AltGr" + "{ " (7:ans tangent).

Del A, Teoriuppgifter

Svara, på separat papper, kort men koncist på frågorna. På fritextfrågor ska svaren motiveras. Otydliga lösningar, både vad gäller läsbarhet och begriplighet, rättas inte.

Varje uppgift kan ge maximalt 2 poäng. I uppgifterna 1-4 är noll till flera alternativ korrekta.

1. Vilka av nedanstående påståenden om nyckelordet `static` är korrekta i språket Java?
 - A. En datamedlem deklarerad `static` kan endast tilldelas ett värde en gång
 - B. En datamedlem deklarerad `static` har klass-räckvidd, dvs. är gemensam för alla objekt av klassens typ
 - C. En metod deklarerad `static` kan anropas utan en objektreferens
 - D. En metod deklarerad `static` får inte referera "vanliga" datamedlemmar (instansvariabler, dvs. datamedlemmar som inte är deklarerade `static`)
2. Vilka av nedanstående påståenden är sanna för nyckelordet `final` i Java?
 - A. Värdet på en variabel av primitiv typ (`int`, `double` etc) som deklarerats `final` får inte ändras efter första tilldelningen
 - B. Om ett objekt refereras av en referens som deklarerats `final` kan objektets tillstånd fortfarande ändras
 - C. Om en metod deklarerats `final` får implementationen av metoden inte ändra i objektets tillstånd
 - D. Om en klass deklarerats `final` får inga subklasser till klassen skapas
3. Antag att vi har en klass, `Foo`, som används av andra klasser. Vilka av nedanstående ändringar i `Foo` innebär att andra klasser som redan använder funktionaliteten i `Foo` kan behöva uppdateras?
 - A. Publika metoder läggs till i `Foo`
 - B. Den inre implementationen av en publik metod i `Foo` ändras, men metodens kontrakt och signatur (namn, returtyp och argumentlista) ändras inte
 - C. Namn på privata metoder i `Foo` ändras
 - D. Argumentlistan för en publik metod i `Foo` ändras, men metodens namn ändras inte
4. En list-klass kan implementeras med en intern privat array eller som en så kallad länkad lista. I vilka av situationerna nedan har en länkad lista bättre prestanda än en lista med en intern array?
 - A. Om man ofta försöker accessa ett godtyckligt element i listan, utan att ta bort elementet
 - B. Om man ofta lägger till ett element på en godtycklig position i listan
 - C. Om man ofta tar bort ett element från en godtycklig position i listan
 - D. Om man ofta itererar över hela listan
5. Vad innebär det att omdefiniera en metod (override) i en klass? Beskriv en konkret situation där det finns en poäng att göra en sådan omdefiniering
6. Beskriv designmönstret Facade (fasad) och en situation där det är meningsfullt att använda mönstret. Är Facade främst ett exempel på hög kohesion eller på låg koppling?

7. Metoden nedan ska öppna och läsa in en text från en fil, men om något fel uppstår signalera detta till anropande metod. Vad är problemet med metoden nedan vad gäller felhantering? Hur borde metoden implementeras i stället?

```
public String readText(File file) {
    BufferedReader reader = null;
    try {
        reader = new BufferedReader(new FileReader(file));
        String data;
        // read text from file into data
        return data;
    }
    catch(IOException e) {
        System.out.println("Sorry, something went wrong. " + e.toString());
    }
    finally {
        if(reader != null) reader.close();
    }
    return null;
}
```

8. Objekten i ett filsystem kan beskrivas med en supertyp, Node, och subtyperna File respektive Directory (mapp). Ett objekt av typen Directory kan innehålla andra objekt av subtyper till Node. Rita ett klassdiagram, enligt UML, som beskriver detta.

Del B, Programmeringsuppgifter

Lösningar lämnas i ett gemensamt projekt på tentamenskontot, enhet H. För varje lösning ska det finnas ett paket med namnet "uppg9", "uppg10" etc. Javas API-dokumentation finns på P:/Java/api.

Alla klasser i denna del av tentamen ska följa objektorienterade principer, speciellt inkapsling. Om inte annat anges ska alla klasser ha meningsfulla konstruktorer, som korrekt initierar alla datamedlemmar, korrekta access-metoder samt en omdefinierad version av toString.

NB! Var försiktig med "Generate" i IntelliJ, det är inte säkert att den autogenererade koden uppfyller kontraktet för klassen du skriver.

9. Vektor

Skriv en klass, Vector3D, som representerar en vektor med tre flyttal (x, y respektive z).

I kontraktet för klassen ingår att objekten ska vara "immutable", dvs tillståndet för ett objekt ska inte kunna ändras efter att objektet skapats. Den inre implementationen kan antingen göras med tre separata datamedlemmar eller med en array. Du väljer själv inre implementation, men måste naturligtvis vara konsekvent genom hela klassen.

Du kommer även att behöva en enum, Index, med värdena X, Y, Z.

A) Nedanstående funktionalitet ska implementeras i klassen Vector3D.

- Två konstruktorer, en som tar tre doubles som argument och en som tar en array av doubles som argument. I det senare fallet ska ett exception kastas om arrayen inte har längden tre,

- en metod, `getValues()`, som returnerar en ny double-array med värdena på x, y och z,
- en metod, `get(Index)`, som tar ett värde av enum-typen `Index` som argument och returnerar motsvarande datamedlem,
- en metod, `length`, som returnerar vektorns längd, $\sqrt{x^2 + y^2 + z^2}$ (använd `java.lang.Math`),
- en `toString`-metod som returnerar textsträng med formatet "[x, y, z]".

Skriv en main-metod där du skapar objekt och demonstrerar funktionaliteten.

B) Lägg till två metoder, `add`, för att addera vektorer. Båda metoderna ska skapa och returnera ett nytt objekt. Den ena metoden ska vara en klassmetod, den andra en vanlig metod (instansmetod). Den senare metoden adderar `this` med ett annat objekt. Du ska bara behöva skriva detaljer om additionen i en av metoderna, valfritt vilken, den andra metoden ska anropa den första.

Demonstrera den nya funktionaliteten i din main-metod.

(4+2 p)

10. Unik lista

Skriv en klass, `UList`, som representerar en lista där elementen är unika, dvs. listan får inte innehålla dubletter. Listklassen ska vara *generisk* och elementen ska internt lagras i en *privat array*.

A) Listklassen ska ha nedanstående funktionalitet (E betecknar typparametern).

- En parameterlös konstruktor, som skapar en intern array med längden 10,
- `size`, som returnerar aktuellt antal element i listan,
- `E get(int index)`, som returnerar en referens till elementet på angivet index,
- boolean `add(E obj)` som, om objektet inte redan existerar i listan, lägger till objektet sist och returnerar `true`, i annat fall returneras `false`,
- boolean `contains(E obj)` som returnerar `true` om objektet existerar i listan, annars `false`,
- en omdefinierad version av `toString` som ger en textsträng på formen "[obj1, obj2, ...]".

Alla metoder som tar ett index som argument ska först kontrollera om index är felaktigt och om så är fallet kasta ett `IndexOutOfBoundsException`. Metoden `add` ska, då det behövs, utöka den interna arrayen.

Skriv en main-metod där du skapar `UList`-objekt och demonstrerar funktionaliteten.

B) Lägg till funktionalitet för att iterera över objekten i listan med hjälp av en iterator. Klassen `UList` ska implementera interfacet `java.lang.Iterable<E>`. Iteratoren, som skrivs som en inre privat klass, ska implementera interfacet `java.util.Iterator<E>`. Du behöver endast implementera metoderna `hasNext` och `next` från interfacet.

Demonstrera funktionaliteten både med direkt användning av `hasNext` och `next` i en while-loop, och i en for-each-loop.

(4+2 p)

11. Modifiering

Vi vill ha funktionalitet för att kunna iterera över en lista med objekt och generera en ny lista, med samma längd, där varje objekt i den nya listan har skapats genom att en metod har applicerats på motsvarande objekt i den ursprungliga listan. Till exempel ska man utifrån en lista med textsträngar kunna generera en ny lista med textsträngar i versaler, eller utifrån en lista med information om personer kunna generera en lista med personernas namn.

Metoden som genererar den nya listan ska ha den signatur som visas nedan.

```
public static <A, R> List<R> mapTo(List<A> data, Converter<A,R> converter)
```

Notera att metoden har två typparametrar, där A representerar typen på objekten i den ursprungliga listan och R typen i den genererade listan.

Koden för att skapa ett nytt objekt, baserat på ett objekt i den ursprungliga listan, definieras i implementationer av ett interface, Converter, som deklarerar den abstrakta metoden

```
R convert(A arg)
```

Skriv en main-metod där du skapar listor med textsträngar respektive Vector3D-objekt och genererar nya listor med modifierade objekt genom att anropa mapTo. Använd implementationer av interfacet Convert som

- givet en textsträng returnerar motsvarande textsträng med enbart versaler respektive
- givet ett Vector3D-objekt returnerar objektets längd (returtypen är då Double).

Åtminstone en av implementationerna av interfacet ska göras med ett lambda-uttryck.

(4 p)